

Делегування подій, бібліотеки, деструктуризація

Головна ідея: Делегування подій використовує механізм спливання (bubbling), щоб обробляти події групи елементів одним слухачем замість багатьох. Бібліотеки дають готовий, перевірений код, який не треба писати з нуля. Деструктуризація дозволяє "розпакувати" значення з об'єктів і масивів у змінні одним компактним виразом замість багаторазового звернення через крапку чи індекс.

1. Поширення подій (event propagation)

Життєвий цикл будь-якої події складається з трьох фаз:

Фаза	Що відбувається
Capturing (занурення)	подія рухається від window вниз до цільового елемента, крізь усіх предків
Target (таргетинг)	подія досягла елемента, на якому вона фактично сталася
Bubbling (спливання)	подія рухається від цільового елемента назад угору до window

На практиці працюють майже виключно з фазою **bubbling** -- саме на ній за замовчуванням спрацьовують обробники, додані через `addEventListener` .

Спливання (bubbling) на прикладі

```
<div id="parent">
  <div id="child">
    <div id="descendant">...</div>
  </div>
</div>

// Клік на #descendant викличе обробники в порядку:
// descendant -> child -> parent -> ... -> window
```

event.target vs event.currentTarget

Властивість	Значення
<code>event.target</code>	елемент, на якому подія фактично сталася -- незмінний протягом усього спливання
<code>event.currentTarget</code>	елемент, до якого прив'язаний обробник , що зараз виконується -- може відрізнятись від target, якщо обробник висить вище по дереву

Припинення спливання

Метод	Дія
<code>event.stopPropagation()</code>	зупиняє спливання вгору; інші обробники на цьому самому елементі й далі виконуються
<code>event.stopImmediatePropagation()</code>	зупиняє спливання і зупиняє решту обробників на цьому самому елементі, навіть якщо вони зареєстровані раніше

Не зупиняй спливання без потреби -- це може зламати сторонній код, що покладається на спливання (наприклад, вебаналітику).

2. Делегування подій (event delegation)

Замість того щоб додавати обробник на кожен з багатьох однотипних елементів, додається **один обробник на спільного предка**, а завдяки bubbling події від дітей "спливають" до нього.

```
const box = document.querySelector(".box");

box.addEventListener("click", function (event) {
  console.log(event.target); // саме той елемент, на якому клікнули
});
```

Три кроки делегування

- Визначити спільного предка групи елементів
- Зареєструвати на предку один обробник потрібної події
- В обробнику через `event.target` визначити, на якому саме дочірньому елементі відбувся клік

Перевірка цільового елемента

```
function selectColor(event) {
  if (event.target.nodeName !== "BUTTON") {
    return; // клікнули між кнопками -- ігноруємо
  }
  const selectedColor = event.target.dataset.color;
}
```

Перевірка типу цільового елемента (через `nodeName` , `classList.contains` чи `closest()`) обов'язкова -- інакше клік "між" елементами теж спрацює й може викликати помилку при зверненні до неіснуючих даних.

Перевага делегування: не потрібно вручну додавати/видаляти обробники, коли елементи групи динамічно з'являються чи зникають -- один слухач на предку продовжує працювати для всіх, включно з новододаними елементами.

3. Бібліотеки та CDN

Бібліотека -- це набір готового, перевіреного коду (функції, методи, класи) для типових завдань, який не треба писати заново.

Характеристика	Що дає
Багаторазове використання	той самий код інтегрується в різні проекти
Прискорення розробки	готові рішення замість написання й тестування з нуля
Підтримка спільнотю	виправлення багів, оновлення, зворотна сумісність

Приклади: **Chart.js** (графіки), **Lodash** (робота з масивами/об'єктами/рядками).

CDN (Content Delivery Network)

Географічно розподілена мережа серверів, що зберігає копії файлів (скриптів, стилів) ближче до користувача -- це прискорює завантаження і знижує навантаження на вихідний сервер.

```
<!-- Підключення бібліотеки Lodash через CDN -->
<script async src="https://cdn.jsdelivr.net/npm/lodash@4.17.21/lodash.min.js"></script>
<script defer src="path/to/script.js"></script>

console.log(_.sum([4, 2, 8, 6])); // 20 -- _ це глобальний об'єкт Lodash
```

Скрипт бібліотеки підключай **до** свого основного файлу, з атрибутом **async** для якнайшвидшого завантаження. Ім'я глобальної властивості (`_` для Lodash) унікальне для кожної бібліотеки й вказане в її документації.

Як читати документацію бібліотеки

Розділ документації	Що там шукати
"Get Started" / "Installation" / "Setup"	як встановити і підключити бібліотеку
"Usage" / "Examples" / "API"	приклади використання основних функцій
"Options" / "Configuration" / "Advanced"	додаткові налаштування і можливості

4. Деструктуризація об'єктів

```
const user = { name: "Jacob", age: 32 };
const { name, age } = user; // замість user.name, user.age окремо
```

Можливість	Синтаксис
Значення за замовчуванням	<code>const { coverImage = "default.jpg" } = book;</code>
Перейменування змінної	<code>const { author: bookAuthor } = book;</code>
Перейменування + значення за замовчуванням	<code>const { coverImage: img = "default.jpg" } = book;</code>
Глибока (вкладена) деструктуризація	<code>const { stats: { followers } } = user;</code>

Якщо властивості з таким іменем немає в об'єкті -- змінній присвоюється **undefined** (якщо не задано значення за замовчуванням). Помилки при цьому не виникає.

У циклах і параметрах функцій

```
for (const { title, author } of books) {
  console.log(title, author);
}

// Патерн "Об'єкт параметрів" + деструктуризація одразу в сигнатурі
function printUserInfo({ name, age, hobby }) {
  console.log(`Name: ${name}, Age: ${age}, Hobby: ${hobby}`);
}
```

Патерн "Об'єкт параметрів": якщо функція приймає більше 2-3 значень, краще передавати один об'єкт з іменованими властивостями замість довгого списку позиційних аргументів -- код виклику стає самодокументованим.

5. Деструктуризація масивів

```
const color = [200, 255, 100];
const [red, green, blue] = color; // квадратні дужки, порядок має значення
```

Можливість	Синтаксис
Значення за замовчуванням	<code>const [r, g, b, a = 1] = color;</code>
Пропуск елементів	<code>const [, , blue] = rgb;</code> -- пропускає перші два
Збір "решти" в масив	<code>const [first, ...rest] = arr;</code>

```
function printFruits([first, second, third]) {
  console.log(first, second, third);
}
printFruits(["apple", "banana", "orange"]);
```

Практичний прийом: обмін значень змінних

```
let a = 1;
let b = 2;
[a, b] = [b, a]; // обмін без тимчасової змінної
console.log(a, b); // 2 1
```

Права частина спочатку створює тимчасовий масив `[b, a]` зі старими значеннями, і лише потім відбувається присвоєння -- тому це працює навіть без окремої змінної для "буфера". Так само можна міняти місцями 3+ змінні одразу.

Пастка: деструктуризація в **уже оголошені** змінні (без `const` / `let`) потребує дужок навколо всього виразу -- `{ a, b } = obj`; -- інакше JS сприйме `{` як початок блоку коду, а не об'єкта, і видасть помилку.

6. Швидкий довідник

Поняття / синтаксис	Суть
Capturing / Target / Bubbling	три фази поширення події: вниз до цілі, на цілі, назад угору
<code>event.target</code>	де подія фактично сталася (незмінний)
<code>event.currentTarget</code>	де зараз висить обробник, що виконується
<code>event.stopPropagation()</code>	зупинити спливання вгору
<code>event.stopImmediatePropagation()</code>	зупинити спливання + інші обробники на цьому елементі
Event delegation	один обробник на предку замість багатьох на дітях
Бібліотека	готовий, перевірений код для типових завдань
CDN	мережа серверів для швидкої доставки файлів бібліотек
Деструктуризація об'єкта	<code>const { key } = obj;</code> -- за іменем властивості
Деструктуризація масиву	<code>const [a, b] = arr;</code> -- за порядком елементів
Значення за замовчуванням	<code>= значення</code> після імені змінної
Перейменування (об'єкт)	<code>{ oldName: newName }</code>
Rest у деструктуризації	<code>...rest</code> -- збирає залишок в новий об'єкт/масив
Об'єкт параметрів	один об'єкт з іменованими полями замість багатьох позиційних аргументів
Обмін змінних	<code>[a, b] = [b, a]</code> -- без тимчасової змінної